

排序：把一组乱序的数据按照 key 排序

(补充) • 稳定性：若元素序列的两个元素 $r[i]$ 、 $r[j]$ ，其 $k[i]=k[j]$ ，则考察排序前后 $r[i]$ 、 $r[j]$ 的位置关系是否一致，即当前方法是否稳定。

• **插入排序**：每次将一个待排元素插入到适当位置上，直至全部插入

• 直接插入排序：将第 i 个元素插入到已排好的 $i-1$ 个元素序列中(直接--比较) (i 从 $1 \sim n$)

总排序码比较次数 KCN：最好 $n-1$ ，最坏 $\frac{n(n-1)}{2}$ } 平均 $O(n^2)$
总元素移动次数 RMN：最好 0 ，最坏 $\frac{(n+1)(n-1)}{2}$

• 折半插入排序：总原理一致，但将第 i 个元素插入前 $i-1$ 个元素中时用折半搜索

总排序码比较次数 KCN：一定为 $O(n \cdot \log_2 n)$

总元素移动次数 RMN：与直接插入一致

• 希尔排序：思想：缩小增量，优化直接插入

方法：①. 取一个整数 $gap < n$ (取一开始 $gap = \lfloor \frac{n}{2} \rfloor$ 较好)，将全部元素分为 gap 个子序列，(即：所有下标模 gap 同余的元素在同一个子序列中)，然后在每个子序列中直接插入排序。

② 缩小 gap (取 $gap = \lfloor \frac{gap}{3} \rfloor + 1$ 较好)，重复上述的划分和排序工作

③ 直至最后 $gap=1$ ，即所有元素在同一个序列中，完成排序。

优点：一开始 gap 大，每组元素少，排得快；后面 gap 小，但元素已基本有序，故排得也快。

(n 较大时，KCN 和 RMN 约在 $n^{1.25} \sim 1.6n^{1.25}$ 之间，因此时间复杂度较低)

eg: 对 21, 25, 49, 25*, 16, 8 排序:



• **快速排序**

思想：任取待排序元素序列中的某个元素(例如：第1个元素)作为基准；根据其 key 将其放到合适的位置，使得整个元素序列划分为左右两个子序列，左边所有元素的 key 均小于基准元素；右边均大于。然后对左右递归即可。

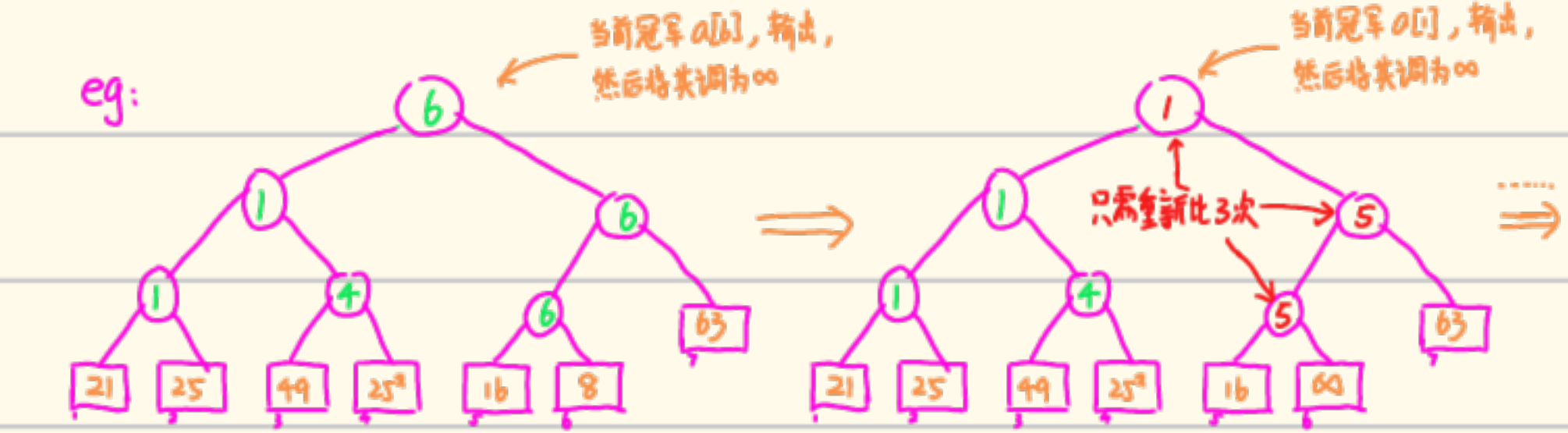
* 将 pivot 放到合适位置的方法：

设置 pivotplace 指针(初始为0)；然后 i 从 $1 \sim n-1$ 每个元素和 pivot 比较，若大于 pivot 则不操作，否则将 pivotplace + 1，然后将 $list[i]$ 和 $list[pivotplace]$ 作交换(即：小的换到前面来)；最后将 $list[0]$ 和 $list[pivotplace]$ 交换。(pivot 元素就位)

• **选择排序**

• 直接选择排序：第 i 轮中(i 从 $0 \sim n-1$)，遍历 $list$ 找到第 i 小的(即下标从 i 到 $n-1$ 遍历)，然后将其与 $list[i]$ 交换(即“选出当前最小的就位”)

• 锦标赛排序： n 个元素作为完全二叉树的 n 个叶节点，各父节点为双亲中较小的，故树根即为当前最小的元素(此树即为胜者树)。每次输出冠军后，将其改为 ∞ ，然后只需重新比其祖先这一路即可。



• 堆排序：利用最大堆实现排序

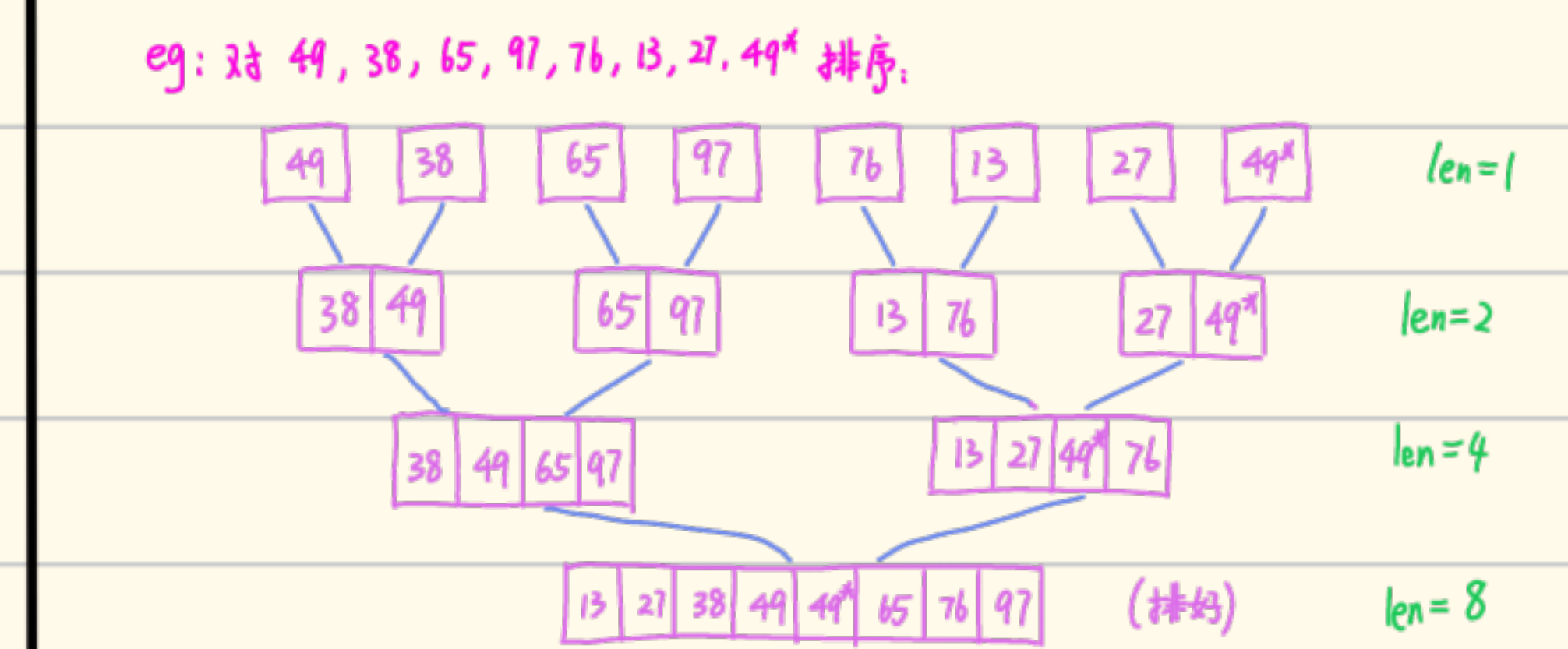
① 初始序列通过 sift down 操作，形成最大堆

② 第 i 轮：(i 从 $1 \sim n$) 将堆顶 ($heap[0]$ ，即当前最大元素)与 $heap[n-i]$ 交换，然后对 $heap[0]$

做 sift down 操作(堆大小为 $n-i$ ，即换下的大元素被“固定”了)，然后 $i++$ 重复执行。
↓
因此在原数组上即可操作，不要额外空间

• **归并排序**

• 归并：元素序列 L_1 中有 2 个有序表 $Vector[left, \dots, mid]$ 和 $Vector[mid, \dots, right]$ ，其可以归并成一个有序表 $Vector[left, \dots, right]$ (用两个指针依次从左向右检测即可实现)



• **总结：**

排序方法	比较次数 KCN		移动次数 RMN		稳定性	附加存储
	最好	最差	最好	最差		
直接插入	$O(n)$	$O(n^2)$	0	$O(n^2)$	✓	$O(1)$
折半插入	$O(n \log_2 n)$		0	$O(n^2)$ (时间也为 $O(n^2)$)	✓	$O(1)$
冒泡	$O(n)$	$O(n^2)$	0	$O(n^2)$	✓	$O(1)$
快排	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(n^2)$	✗	最好 $O(\log_2 n)$ 最坏 $O(n)$
直接选择	$O(n^2)$		0	$O(n)$	✗	$O(1)$
锦标赛	$O(n \log_2 n)$		$O(n \log_2 n)$		✓	$O(n)$
堆排序	$O(n \log_2 n)$		$O(n \log_2 n)$		✗	$O(1)$
二路归并	$O(n \log_2 n)$		$O(n \log_2 n)$		✓	$O(n)$